

ECS Voxel Platformer

This project takes the existing platformer project and recreates the backend for more modularity, readability and expandability. The project also introduces networking and web based builds. The Projects are:

- **Client:** Client only version of game
- **Client-Web:** Web hosted version of the game to run with server
- **Client-Network:** Client program to run with server
- **Server:** Server program for the networked clients

CoreLib is the static library that contains the shared logic for all the programs.

When starting Client-Web and Client-Network you first must enter the address and port of the server that is running the server program.

- You can also just hit enter for the default local address and port for the clients and server

When starting the server, you can enter a custom port or hit enter for the default port.

Building/Running the Project

Each project builds into the following directory structure:

build/bin/<Debug or Release>/<ProjectName>

For example:

build/bin/Debug/Client

build/bin/Release/Server

The solution can be built using Visual Studio 2026.

Requirements

- Visual Studio 2026
- Emscripten (required for the web client)

The main projects that can be built with VS are:

- Client – Local desktop version of the game with local saves
- Client-Network – Desktop client that connects to the server
- Server – WebSocket server used for save and chunk storage

Web Build

The web client requires Emscripten. Do not build using VS

Navigate to the Client-Web directory and run:

`build_web.bat`

This will build the folder that contains the index.html file used to run the program in a browser. A local web server can be used to launch it can be hosted by a remote server.

Project Info

Levels JSON Structure:

Top-level JSON must contain a **non-empty** array:

- **levels:** [] (required; must not be empty)
- Each item in **levels** is a **level definition object**.

Fields

- **name** (string)
- **exit** (vec3)
- **playerDistance** (num)
- **playerTimer** (int)
- **spawns** (array of "spawn objects")

Spawn objects (what actually gets placed into the level)

- **prefab** (string) **or scene** (string)
- **position** : vec3
- **rotation** : quaternion array [**w,x,y,z**] (explicitly validated as wxyz here)
- **lookAtDirection** : vec3
- **platform** : object (platform motion tuning for that spawned prefab)
- **turret** : object (turret tuning for that spawned prefab)

platform object (if present):

- Required vec3: **from**, **to**, **center**
- Optional numbers: **speed**, **speedMultiplier**

turret object (if present):

- Optional numbers: **delay**, **speed**

Prefabs (from .scn to json)

Prefab JSON structure (what the engine expects)

Top-level:

- **entities**: array (required, non-empty)

Each entry in **entities[]**:

- **id**: string
- **components**: object

Instantiation process:

1. **Create all entities first** (builds a map **id** -> **flecs entity id**)
2. **Apply components** by iterating **components** keys and calling a registered parser

Parent/child relationships (scene graph)

- Parenting is handled through the **transform** component:
- **transform.parent** is a string referencing another entity **id**
- Parenting is applied after all entities are created

Registered in **prefab_json_parsers.hpp**:

- **transform**, **collider**, **renderable**, **light**, **camera**, **behaviour**

Behaviors are authored as:

- **components.behaviour** = { "type": "...", ... }

ECS Structure

The project uses **Flecs** as the ECS backend, with a wrapper in **vpg::ecs**:

- **Entity** is **flecs::entity_t** (an integer id)
- Components are C++ types attached to entities in the Flecs world
- Runtime update is driven by **game::runtime::FlecsRuntime** (created in **CoreLib/src/client_program.cpp**)

World binding (required for everything)

vpg::ecs::bind_world(flecs::world*) sets a global world pointer used by:

- Prefab instantiation (**game::prefab_json::instantiate**)
- **vpg::ecs::create_entity()**

- Component add/remove helpers

If the world isn't bound, entity creation is fatal (aborts), and prefab instantiation fails early.

Entities are created:

- Internally during prefab instantiation via **world->entity().id()**
- Or via **vpg::ecs::create_entity()** (wrapper)

Entities are configured by attaching components

Components:

A component is a data struct (ex: **vpg::ecs::Transform**, **vpg::physics::Collider**, **vpg::gl::Renderable**).

There's also a stream-based component add path in **CoreLib/src/ecs/entity.cpp**:

- Reads a type name from a stream
- Deserializes the matching **Info** struct
- Replaces the component on the entity
(Transform/Behaviour/Collider/Camera/Light/Renderable).

Behaviours (how "scripts" are represented)

Gameplay behaviors are represented as a single component:

- **vpg::ecs::Behaviour**

The prefab JSON "**behaviour**" component does:

- read **type**
- find a parser for that type
- parser builds an **Info** for a concrete behavior type and stores it inside **vpg::ecs::Behaviour** via **vpg::ecs::Behaviour::Info::create<T>(...)**

Adding components and system:

Add a new prefab entity:

- Add another entry in prefab **entities[]** with a unique **id** and a **components** object.
- Add a **transform** if it must exist spatially or be parented.

Add a new JSON component key

Implement a parser with signature:

- **ComponentParser = bool (*)(Entity, JsonValue, EntityLookup, PendingParents, error)**
- Register it in **game::prefab_json_parsers::register_default_parsers()** using **register_component_parser("yourKey", yourParser)**
- Use it in prefab JSON under **components.yourKey**.
-

Add a new behaviour type

- Implement the behaviour type (and its **Info** data).
- Implement a behaviour parser: **BehaviourParser = bool (*)(Entity, JsonValue, EntityLookup, error)**
- Register it in **register_default_parsers()** using **register_behaviour_parser(YourBehaviour::TypeName, yourParser)**